JavaScript is disabled on your browser.

Java SE 27 & JDK 27

Interface CharSequence

All Known Subinterfaces:

`Name`, `Utf8Entry`

All Known Implementing Classes:

`CharBuffer`, `Segment`, `String`, `StringBuffer`, `StringBuilder`

`public interface` **`CharSequence`**

A `CharSequence` is a readable sequence of `char` values. This interface provides uniform, read-only access to many different kinds of `char` sequences. A `char` value represents a character in the *Basic Multilingual Plane (BMP)* or a surrogate. Refer to Unicode Character Representation for details.

This interface does not refine the general contracts of the `equals` and `hashCode` methods. The result of testing two objects that implement `CharSequence` for equality is therefore, in general, undefined. Each object may be implemented by a different class, and there is no guarantee that each class will be capable of testing its instances for equality with those of the other. It is therefore inappropriate to use arbitrary `CharSequence` instances as elements in a set or as keys in a map.

Since:

1.4

Method Summary

All Methods	Static Methods	

Modifier and Type	Method	Description
`char`	**`charAt`**`(int index)`	Returns the `char` value at the specified index.
`default IntStream`	**`chars`**`()`	Returns a stream of `int` zero-extending the `char` values from this sequence.
`default IntStream`	**`codePoints`**`()`	Returns a stream of code point values from this sequence.
`static int`	**`compare`**`(CharSequence cs1, CharSequence cs2)`	Compares two `CharSequence` instances lexicographically.

default void	**getChars**(int srcBegin, int srcEnd, char[] dst, int dstBegin)	Copies characters from this sequence into the given destination array.
default boolean	**isEmpty**()	Returns true if this character sequence is empty.
int	**length**()	Returns the length of this character sequence.
CharSequence	**subSequence**(int start, int end)	Returns a CharSequence that is a subsequence of this sequence.
String	**toString**()	Returns a string containing the characters in this sequence in the same order as this sequence.

Method Details

length

`int length()`

Returns the length of this character sequence. The length is the number of 16-bit chars in the sequence.

Returns:

the number of chars in this sequence

charAt

`char charAt(int index)`

Returns the char value at the specified index. An index ranges from zero to `length() - 1`. The first char value of the sequence is at index zero, the next at index one, and so on, as for array indexing.

If the char value specified by the index is a surrogate, the surrogate value is returned.

Parameters:

index - the index of the char value to be returned

Returns:

the specified `char` value

Throws:

`IndexOutOfBoundsException` - if the `index` argument is negative or not less than `length()`

isEmpty

```
default boolean isEmpty()
```

Returns `true` if this character sequence is empty.

Implementation Requirements:

The default implementation returns the result of calling `length() == 0`.

Returns:

`true` if `length()` is `0`, otherwise `false`

Since:

15

subSequence

```
CharSequence subSequence(int start,
                         int end)
```

Returns a `CharSequence` that is a subsequence of this sequence. The subsequence starts with the `char` value at the specified index and ends with the `char` value at index `end - 1`. The length (in `char`s) of the returned sequence is `end - start`, so if `start == end` then an empty sequence is returned.

Parameters:

`start` - the start index, inclusive

`end` - the end index, exclusive

Returns:

the specified subsequence

Throws:

`IndexOutOfBoundsException` - if `start` or `end` are negative, if `end` is greater than `length()`, or if `start` is greater than `end`

toString

```
String toString()
```

Returns a string containing the characters in this sequence in the same order as this sequence. The length of the string will be the length of this sequence.

Overrides:

`toString` in class `Object`

Returns:

a string consisting of exactly this sequence of characters

chars

```
default IntStream chars()
```

Returns a stream of `int` zero-extending the `char` values from this sequence. Any char which maps to a surrogate code point is passed through uninterpreted.

The stream binds to this sequence when the terminal stream operation commences (specifically, for mutable sequences the spliterator for the stream is *late-binding*). If the sequence is modified during that operation then the result is undefined.

Returns:

an IntStream of char values from this sequence

Since:

1.8

codePoints

```
default IntStream codePoints()
```

Returns a stream of code point values from this sequence. Any surrogate pairs encountered in the sequence are combined as if by Character.toCodePoint and the result is passed to the stream. Any other code units, including ordinary BMP characters, unpaired surrogates, and undefined code units, are zero-extended to `int` values which are then passed to the stream.

The stream binds to this sequence when the terminal stream operation commences (specifically, for mutable sequences the spliterator for the stream is *late-binding*). If the sequence is modified during that operation then the result is undefined.

Returns:

an IntStream of Unicode code points from this sequence

Since:

1.8

compare

```
static int compare(CharSequence cs1,
                   CharSequence cs2)
```

Compares two `CharSequence` instances lexicographically. Returns a negative value, zero, or a positive value if the first sequence is lexicographically less than, equal to, or greater than the second, respectively.

The lexicographical ordering of `CharSequence` is defined as follows. Consider a `CharSequence` *cs* of length *len* to be a sequence of char values, *cs[0]* to *cs[len-1]*. Suppose *k* is the lowest index at which the corresponding char values from each sequence differ. The lexicographic ordering of the sequences is determined by a numeric comparison of the char values *cs1[k]* with *cs2[k]*. If there is no such index *k*, the shorter sequence is considered lexicographically less than the other. If the sequences have the same length, the sequences are considered lexicographically equal.

Parameters:

`cs1` - the first `CharSequence`

`cs2` - the second `CharSequence`

Returns:

the value `0` if the two `CharSequence` are equal; a negative integer if the first `CharSequence` is lexicographically less than the second; or a positive integer if the first `CharSequence` is lexicographically greater than the second.

Since:

11

getChars

```
default void getChars(int srcBegin,
                      int srcEnd,
                      char[] dst,
                      int dstBegin)
```

Copies characters from this sequence into the given destination array. The first character to be copied is at index `srcBegin`; the last character to be copied is at index `srcEnd-1`. The total number of characters to be copied is `srcEnd-srcBegin`. The characters are copied into the subarray of `dst` starting at index `dstBegin` and ending at index:

```
dstbegin + (srcEnd-srcBegin) - 1
```

Implementation Requirements:

The default implementation invokes `charAt(int index)` in a loop iterating `index` from `srcBegin` to `srcEnd-1`. Concurrent truncation of this character sequence can throw `IndexOutOfBoundsException`. In this case, some characters, but not all, may be already transferred.

Parameters:

`srcBegin` - start copying at this offset.

`srcEnd` - stop copying at this offset.

`dst` - the array to copy the data into.

`dstBegin` - offset into `dst`.

Throws:

`IndexOutOfBoundsException` - if any of the following is true:

- `srcBegin` is negative
- `dstBegin` is negative
- the `srcBegin` argument is greater than the `srcEnd` argument.
- `srcEnd` is greater than `this.length()`.
- `dstBegin+srcEnd-srcBegin` is greater than `dst.length`

`NullPointerException` - if `dst` is `null`

Since:

25

Report a bug or suggest an enhancement
For further API reference and developer documentation see the Java SE Documentation, which contains more detailed, developer-targeted descriptions with conceptual overviews, definitions of terms, workarounds, and working code examples. Other versions.
Java is a trademark or registered trademark of Oracle and/or its affiliates in the US and other countries.
Copyright © 1993, 2026, Oracle and/or its affiliates, 500 Oracle Parkway, Redwood Shores, CA 94065 USA.
All rights reserved. Use is subject to license terms and the documentation redistribution policy.

Scripting on this page tracks web page traffic, but does not change the content in any way.